

breadth-first search

CSCI
270

Search $(M = (Q, \Sigma, \Delta, q_0, F, \omega), H)$:

- ▶ **container** = new Container()
- ▶ **container.put**($\langle q_0, 0, H(q_0) \rangle$)
- ▶ **repeat**:
 - ▶ if **container.empty**() then return ∞
 - ▶ **n** = **container.get**()
 - ▶ if $q(n) \in F$ then return $g(n)$
 - ▶ for each transition $(q(n), \sigma, q') \in \Delta$:
 successor = $\langle q', g(n) + \omega(q(n), \sigma, q'), H(q') \rangle$
 container.put(**successor**)

"visit"
a node

what
am i?



Search $(M = (Q, \Sigma, \Delta, q_0, F, \omega), H)$:

- ▶ **container** = new Container()
- ▶ **container.put**($\langle q_0, 0, H(q_0) \rangle$)
- ▶ repeat:
 - ▶ if **container.empty**() then return ∞
 - ▶ **n** = **container.get**()
 - ▶ if $q(n) \in F$ then return $g(n)$
 - ▶ for each transition $(q(n), \sigma, q') \in \Delta$:
 successor = $\langle q', g(n) + \omega(q(n), \sigma, q'), H(q') \rangle$
 container.put(**successor**)

"visit"
a node

a **container**
is a data structure
that supports
these operations

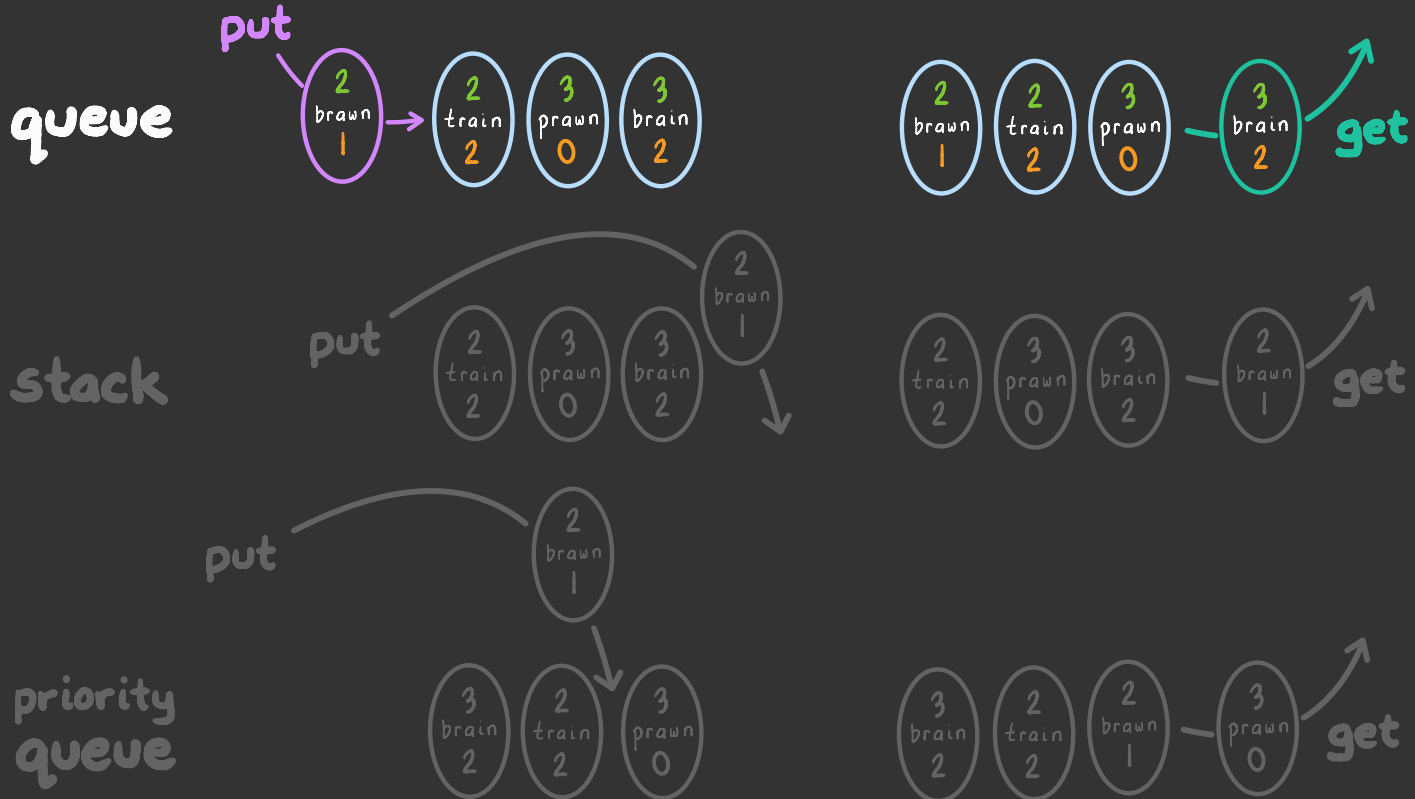
what
am i?



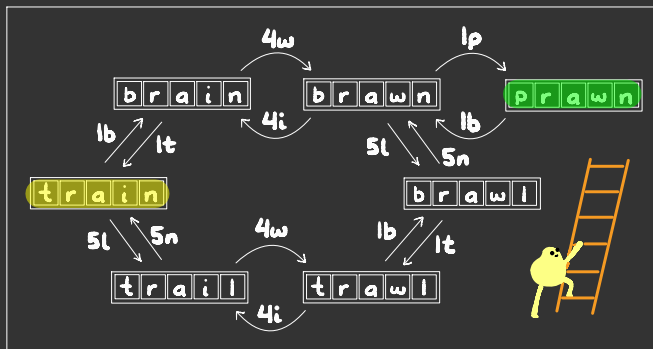
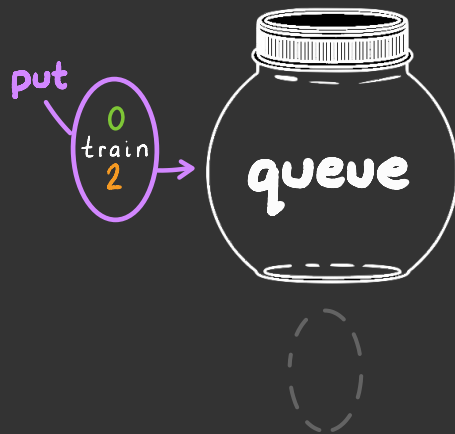
examples of containers



let's try a queue!



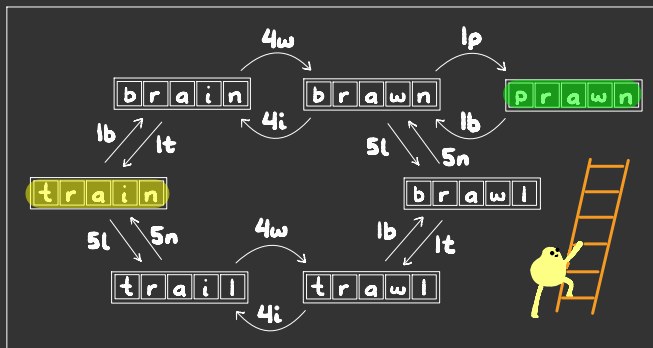
0
train
2

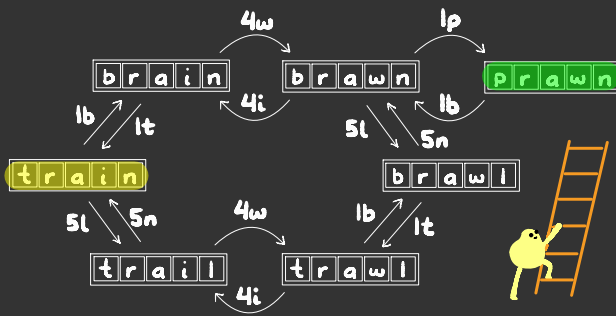
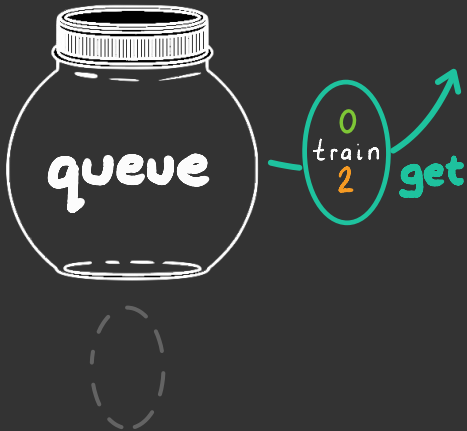


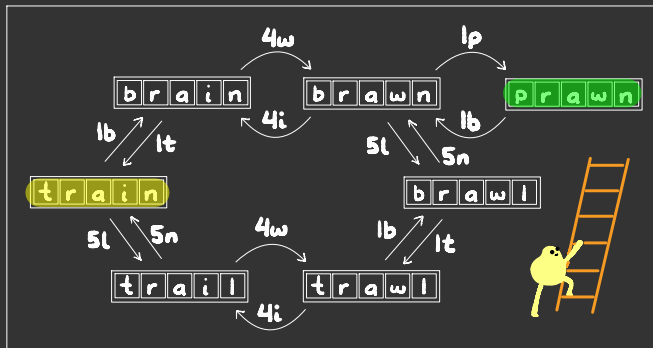
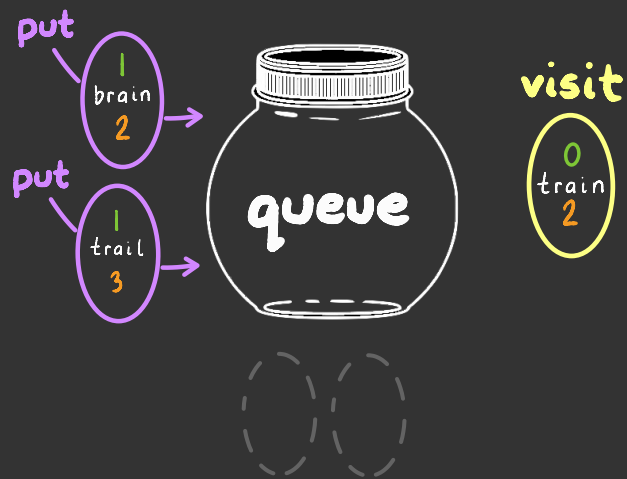
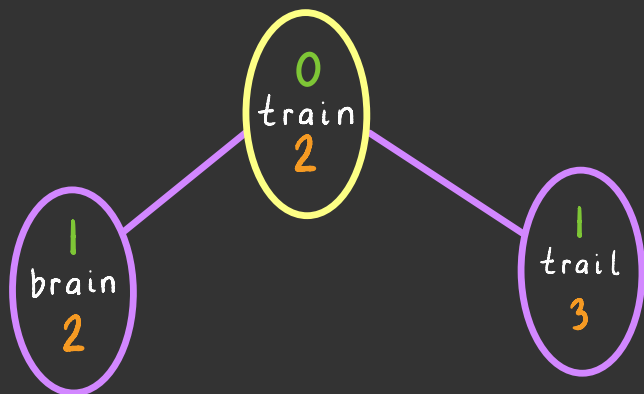
0
train
2

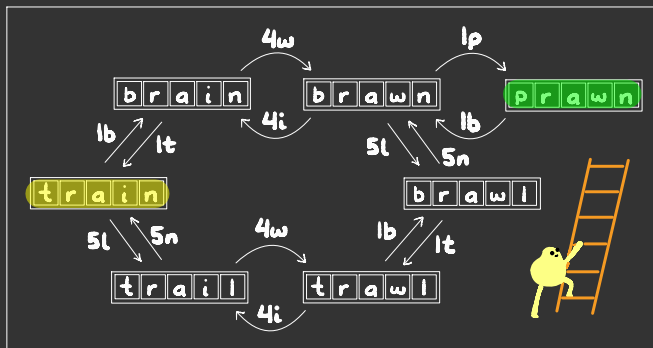
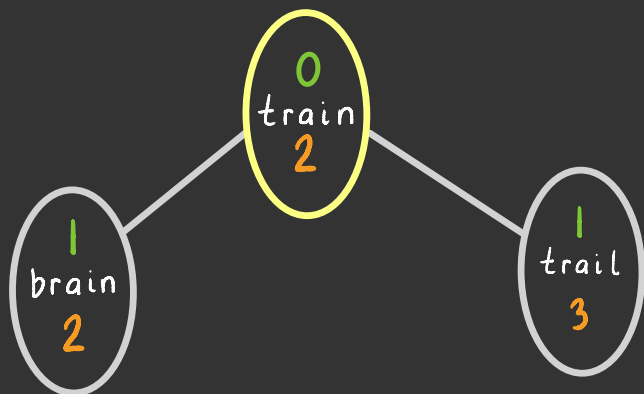


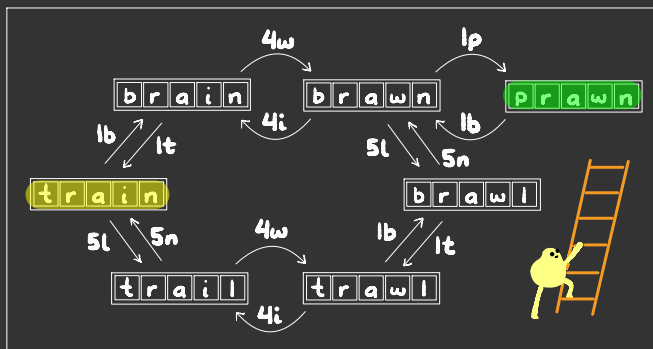
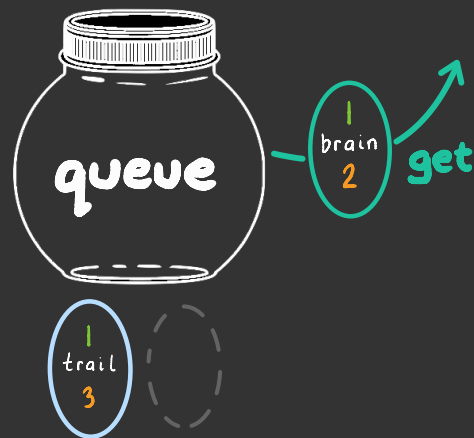
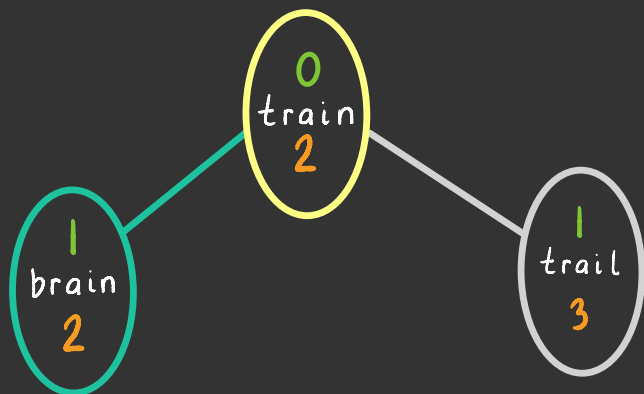
0
train
2

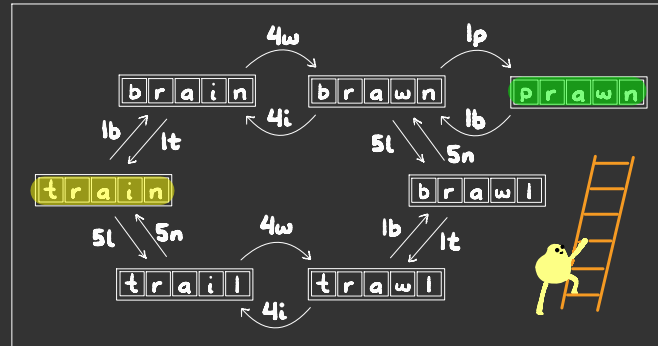
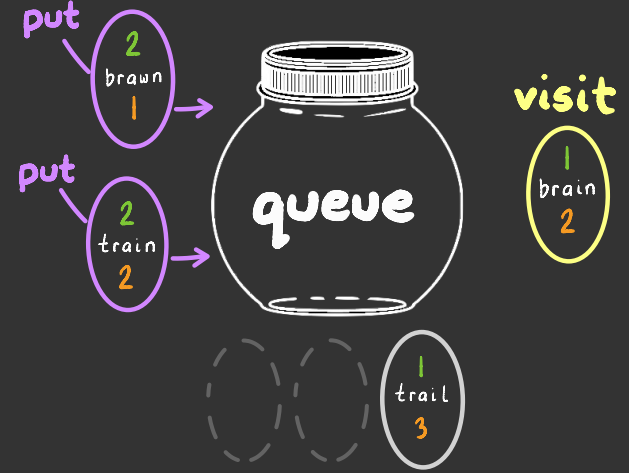
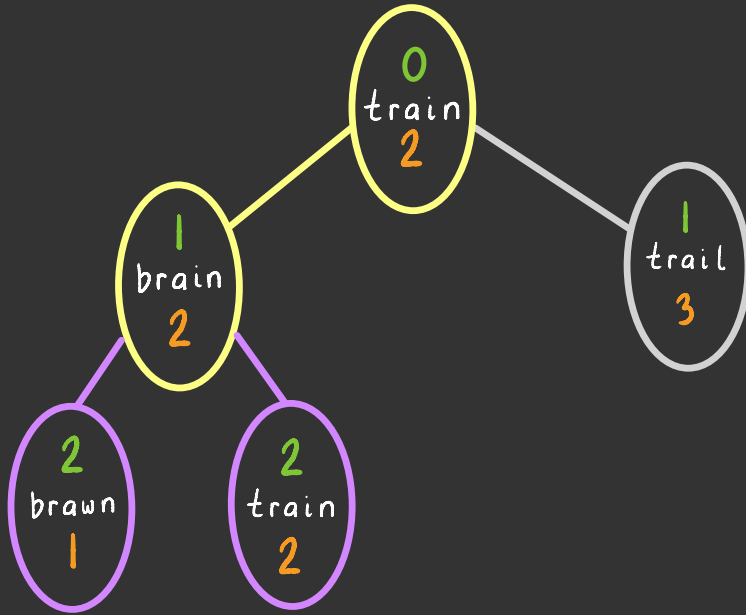


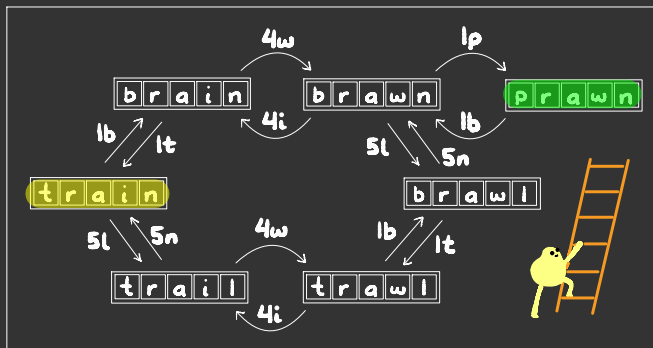
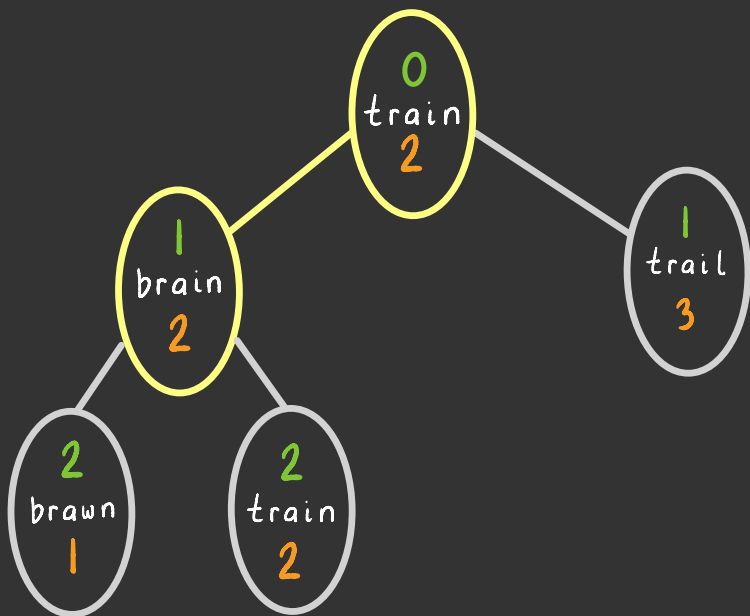


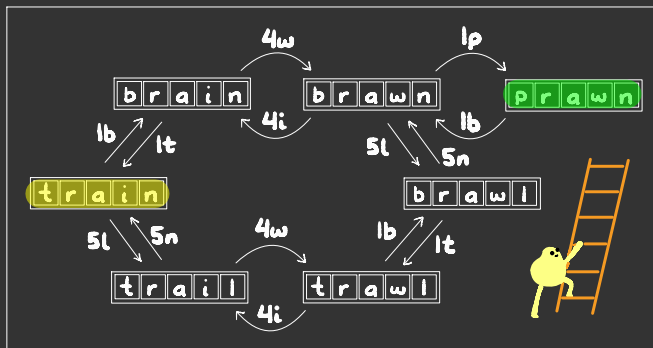
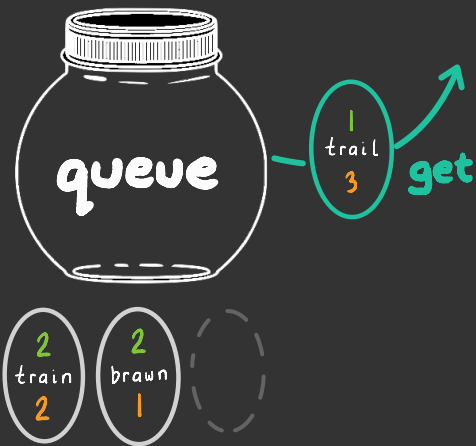
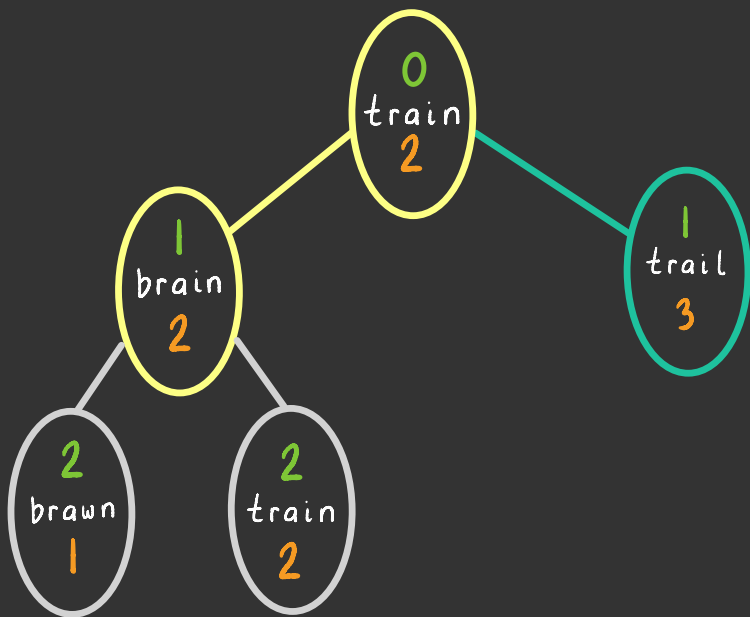


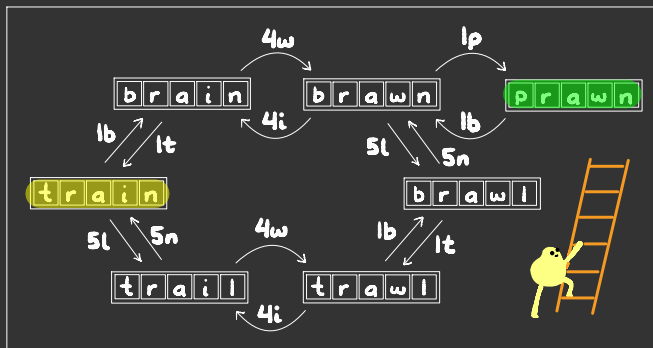
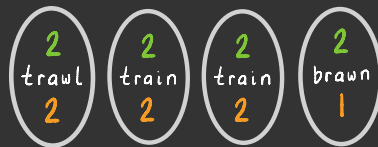
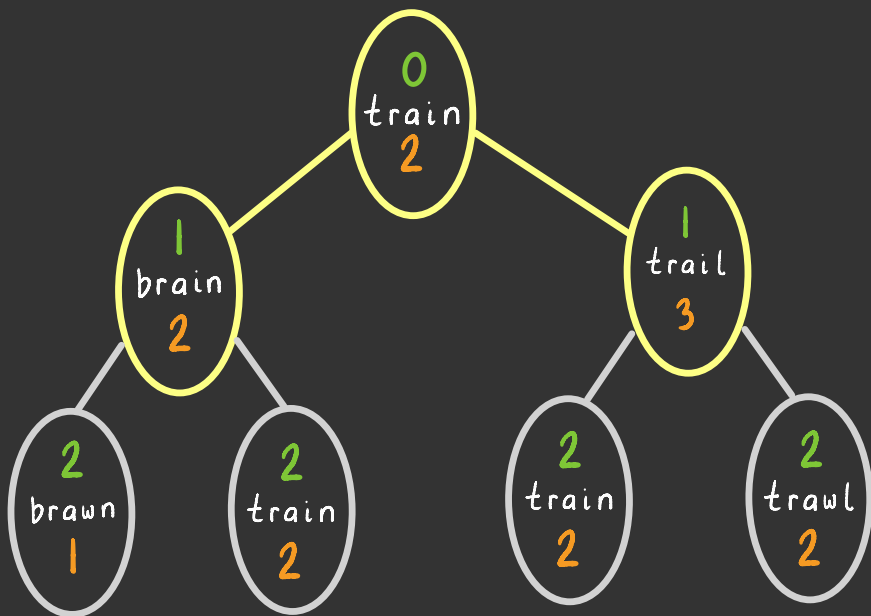


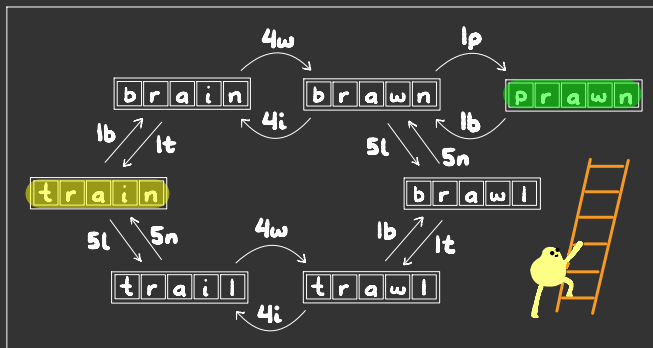
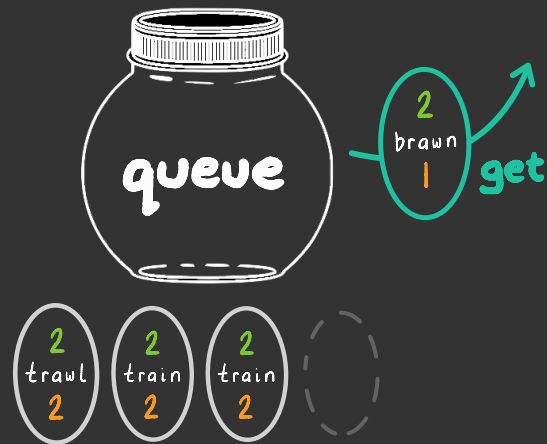
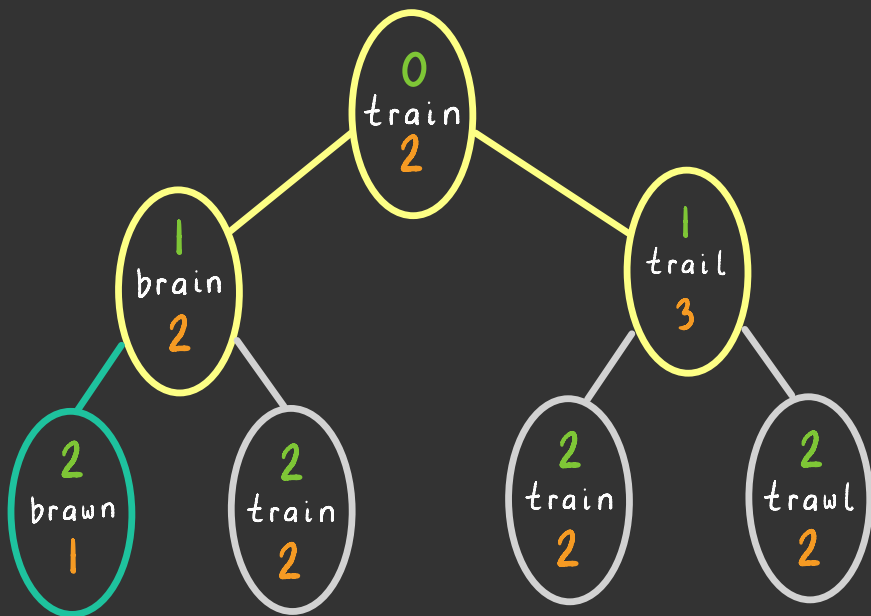


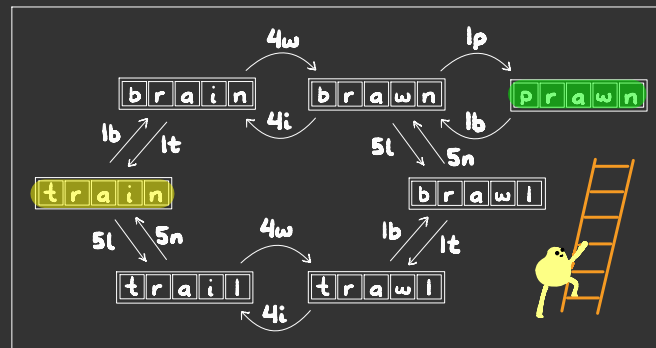
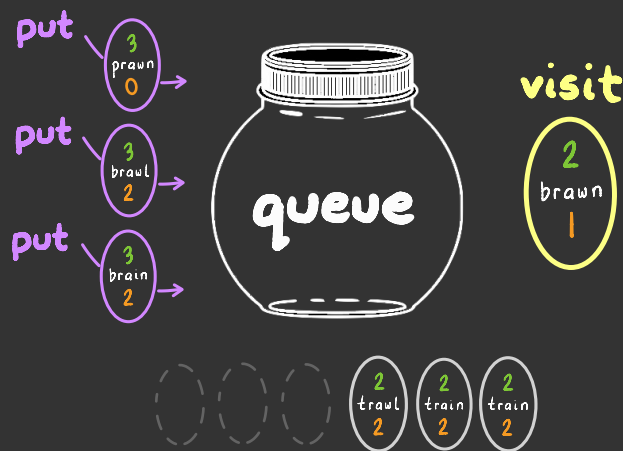
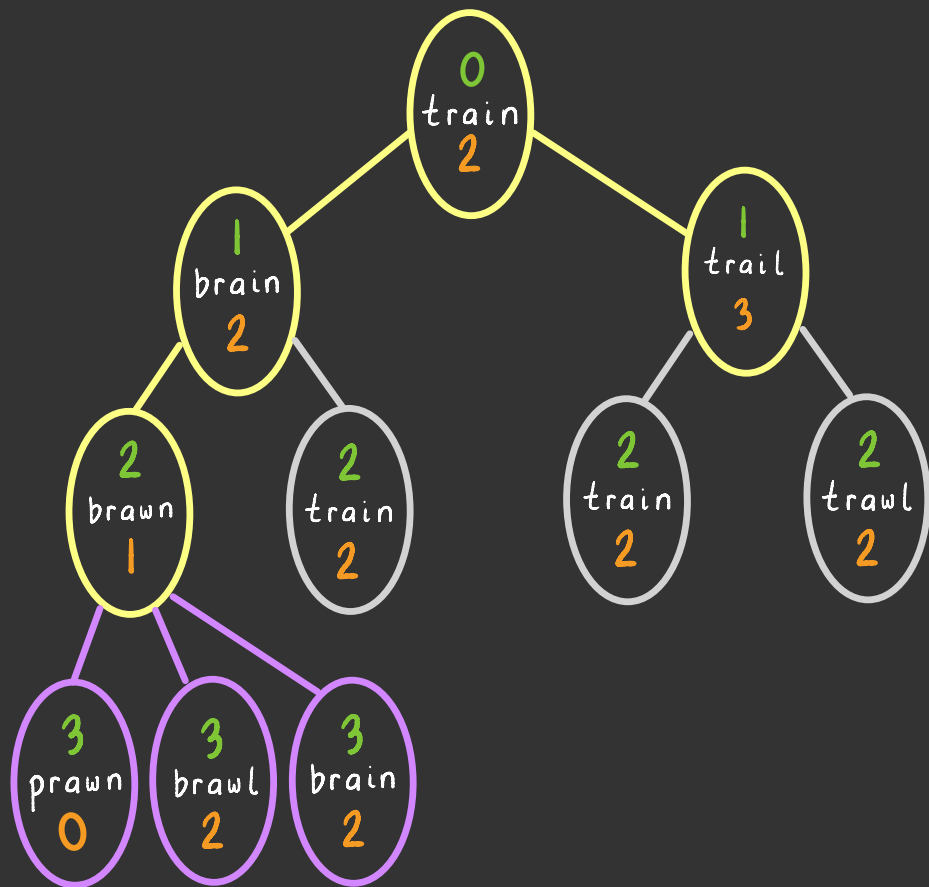


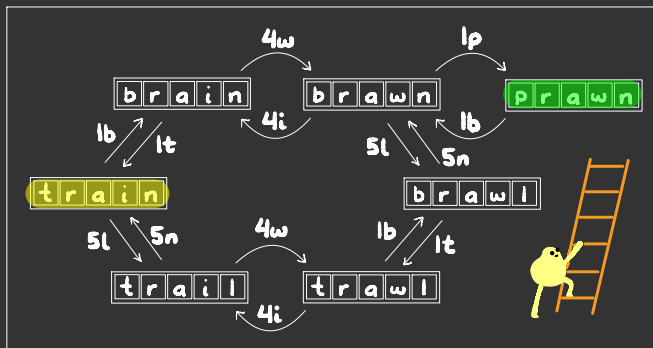
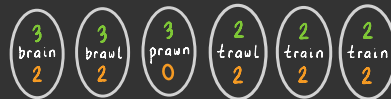
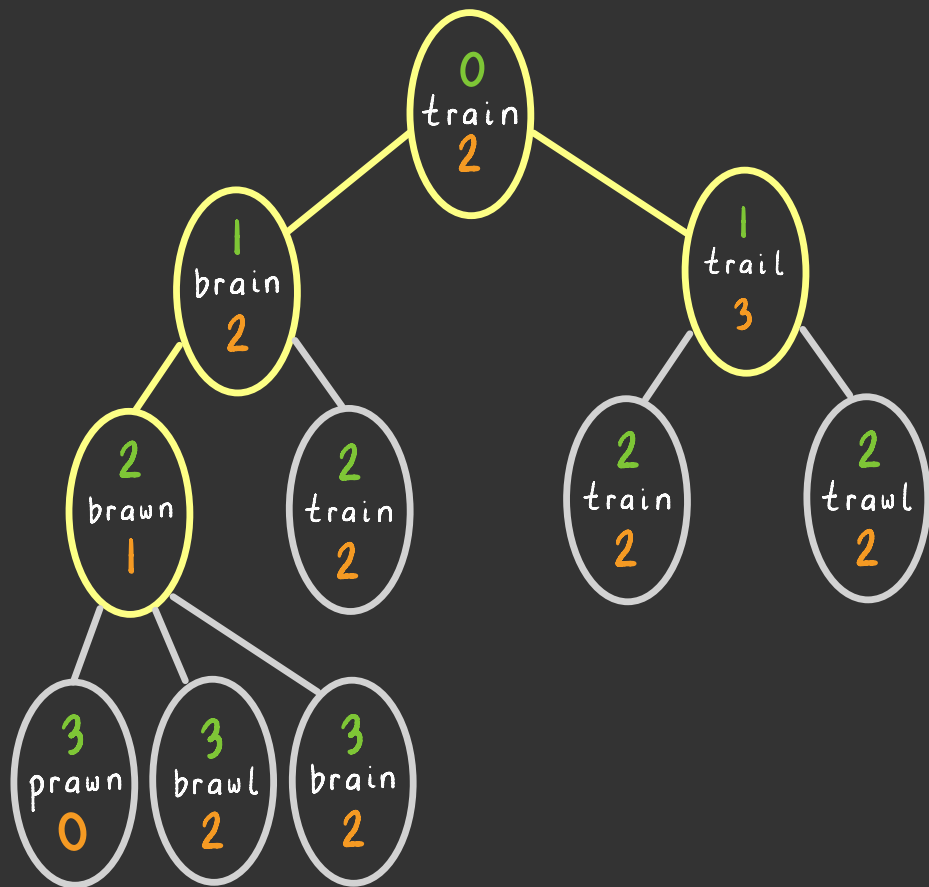


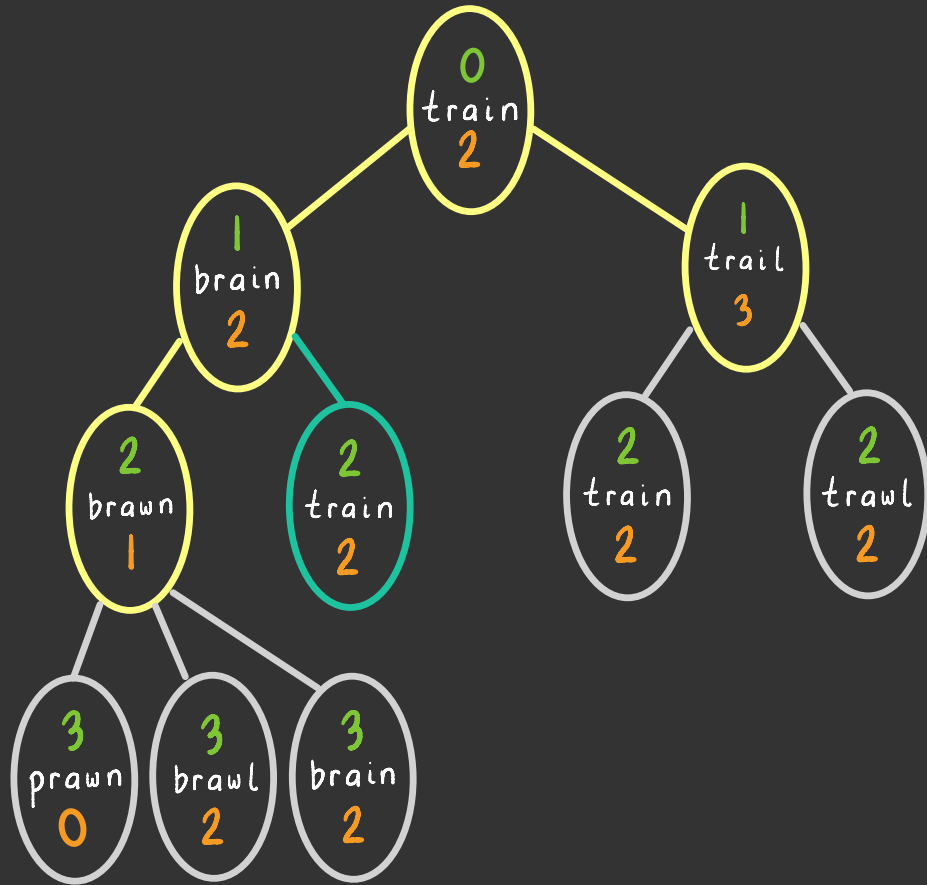






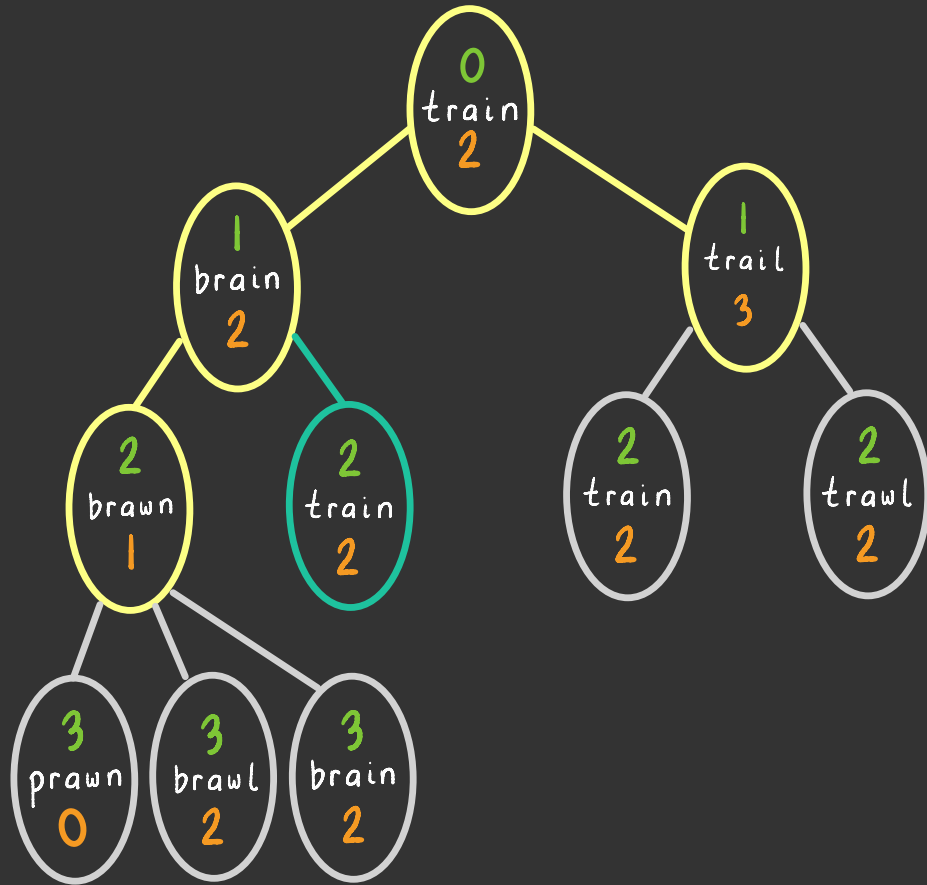






in what order are
the nodes visited?





in what order are
the nodes visited?

for $j < k$, every node at
search depth j is visited
before any node
at search depth k

search ($m = (Q, \Sigma, \Delta, q_0, F, \omega), H$) :

- ▶ `container = new Container()`
- ▶ `container.put( $\langle q_0, 0, H(q_0) \rangle$ )`
- ▶ repeat :
 - ▶ if `container.empty()` then return ∞
 - ▶ `n = container.get()`
 - ▶ if $q(n) \in F$ then return $g(n)$
 - ▶ for each transition $(q(n), \sigma, q') \in \Delta$:
 - `successor =  $\langle q', g(n) + \omega(q(n), \sigma, q'), H(q') \rangle$`
 - `container.put(successor)`

"visit"
a node

if we use a



as the **container**,
the algorithm is called
breadth-first
search (**bfs**)